

Problem Brief — Claims Exception Triage Assistant

Shilpi Verma · UHC Tech AI Transformation, Cohort 5 · Track 1 · August 2026

The problem

When a claim fails auto-adjudication it lands in an exception queue, and an analyst's first job is not fixing it — it's **figuring out why it's stuck**. Pend codes are often generic or stale (~30% in our modeled queue carry codes like P99 that say nothing), so diagnosis means reading free-text adjudicator notes and manually checking prior-auth registries, claim history, and contract tables. Queues are worked FIFO or by age, so a \$42,000 SLA-breaching surgical claim can wait behind a \$45 office visit that arrived earlier.

Target user: the claims operations analyst working that queue. **Pain:** most of the handle time is diagnosis, and the ordering of work ignores impact.

The solution

An assistant that pre-diagnoses every pended claim — root cause from a fixed 7-item taxonomy, a plain-English summary with **cited evidence**, an urgency score (1–100), a proposed next action, and an owner-team routing — then puts a human in front of every decision. The analyst starts at "verify," not "investigate," and the queue is ordered by impact (urgency, dollar tie-break), not arrival.

Architecture in one line: ingest & validate (pydantic) → PHI-pattern redaction → evidence lookup from source systems → one structured LLM call per claim (Claude Sonnet, taxonomy-constrained tool schema) → deterministic guard layer → ranked queue → review UI with approve/reject/reassign → append-only audit log.

Trust design — the model proposes, code disposes:

- Guard rules can only *escalate*: SLA-breach and high-dollar urgency floors, cause↔queue consistency, evidence-contradiction checks. Unit-tested invariant: the guard never lowers urgency.
- Confidence < 0.65, or a claim whose stated cause contradicts registry evidence, is force-routed to a senior-analyst desk. The model cannot choose that queue itself.
- 100% of proposed actions pass a human approval gate; the system executes nothing.
- Every model input hash, model + prompt version, guard override, and human click is one row in an append-only audit log.

Measured results (synthetic dev set, 50 labeled claims; live Claude calls)

Metric	Rule-based baseline (pend-code lookup + dollars)	This system
Root-cause accuracy	72%	100%
Routing accuracy (correct team)	76%	100%
Critical claims (sev 4–5) in top 10	50%	100%
Urgency ↔ severity rank correlation	0.55	0.81

The baseline is the honest strawman for "why an LLM at all": where pend codes are generic, a lookup table guesses blindly; the model reads the note and checks the evidence. Planted adversarial cases (notes that imply missing prior-auth while the registry holds a valid approval) confirm the model verifies against source systems rather than trusting the note.

Read that 100% with the caveat we found ourselves: prompt versions V1, V2, and V3 all score identically, so **this dev set is saturated** — it can no longer distinguish prompt quality, and further tuning against it would measure noise. The same 50 claims hold the baseline to 72%, so the set is not trivially easy; it is simply no longer discriminating at this model tier. A 25-claim sealed eval set is scored exactly once, after prompt freeze, and reported verbatim.

Scope

In (v1): synthetic queue of 75 claims; per-claim structured triage; guard layer; ranked review UI; eval harness with baseline comparison; audit log. **Out (deliberately):** real adjudication or denial decisions, live system integration, auto-execution of any action, fine-tuning, multi-agent orchestration — each excluded with rationale in the PRD; integration is the handoff owner's roadmap.

Honest constraints: all metrics are on synthetic data whose difficulty we controlled; the time-saving claim is a hypothesis until a shadow-mode pilot measures it on real pends. Production requires a BAA/in-VPC model endpoint and field-level PHI minimization — the prototype's redaction layer then becomes defense-in-depth.